Symbolic Regression with Genetic Programming

CSC 370 HW2

Madeline Shi & Elie Jerjees

Overview

01

Introduction

02

Background

03

Experiments

04

Result

Introduction · Finding f from data

Symbolic regression recovers a formula, not just a fit

01 What we're given

3 datasets × 25,000 points

D1 f(x) · + − × ÷ · integer constants

D2 f(x1, x2, x3) · + − × ÷ · real constants

D3 f(x) · adds sin, exp, log (extra credit)

02 Why it's hard

The search space is **every possible expression**

No fixed model: the **form of f is unknown**

A good fit must **generalize** to unseen data, so we hold out a test set

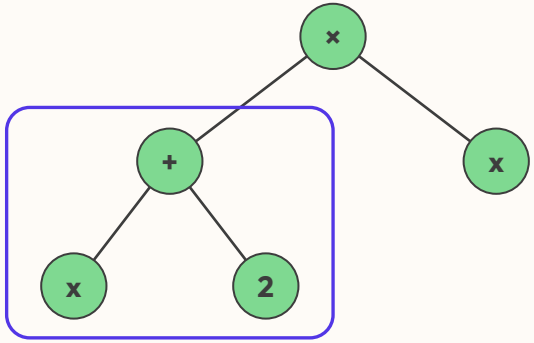
Goal: recover the **form** of f, then check it on held-out data

Background · Genetic programming

Evolve a population of expression trees

01 Trees as individuals

Leaves: variables, constants **Nodes:** + − × ÷



subtree (x + 2)

02 Evolution loop

1. **Initialize:** random trees
2. **Evaluate:** fitness on training data
3. **Select:** tournament
4. **Vary:** crossover + mutation
5. **Repeat:** for N generations

Mutation: replace a subtree **Crossover**: exchange subtrees **Elitism**: keep the best

Background · Overfitting and bloat

Hold out test data and keep trees from growing without limit

01 Avoid overfitting

Train 80% | **Test 20%**

Fitness uses **training data only**

Held out test data **picks the winner** and reports its error

Best on training can fail on unseen data

02 Limit bloat

fitness = mean squared error (lower is better)

Trees grow without limit if unchecked

Depth cap: an offspring deeper than the limit is replaced by its parent

Size penalties are another option (see Dataset 2)

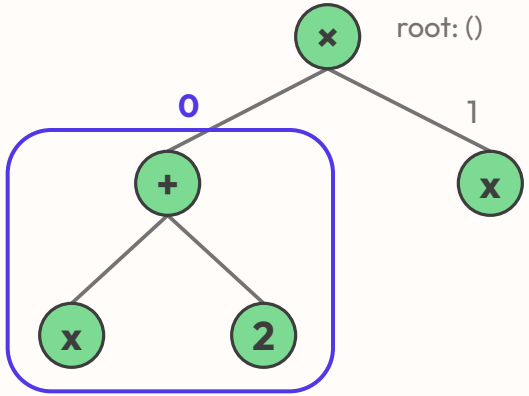
Train/test split guards the result, the depth cap guards the search

Dataset 1 · Basic GP setup

Arithmetic expression trees, subtree variation and MSE selection

01 Trees, paths and subtrees

Terminals: x, integers [−10, 10] **Operators: + − × ÷**



path (0,) → subtree (x + 2)

02 GP settings

100 trees · 50 generations

Initial depth 3 → maximum 5

Early-stop probability: 0.3

Tournament 3 · keep 1 elite

Mutation probability: 0.20

Fitness: training MSE

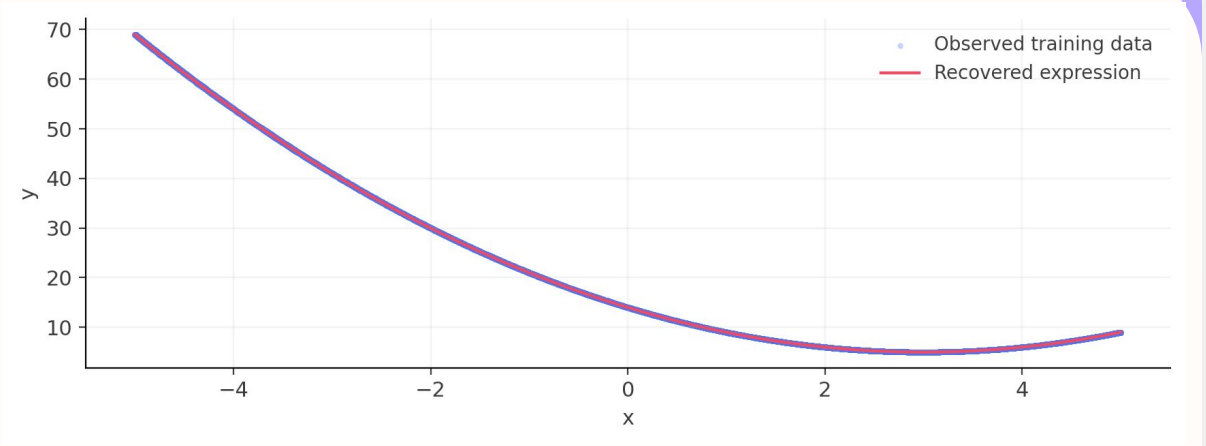
Data split: 2:8

Mutation: replace a subtree **Crossover: exchange subtrees**

Dataset 1 · A compact recovered formula

Population 100 · Generations 50 · seed 42

Observed data and recovered expression



Low train and test errors for the same expression

Recorded result

Training MSE

0.000599

Test MSE

0.000579

Verified from saved terminal output

$$f(x) = x^2 - 6x + 14$$

Dataset 2 · Three variables, huge scales

Same GP engine, adapted to messy experimental data

01 The data

25,000 rows · inputs x1, x2, x3 · output y

x1, x2: about 10^2 to 10^7

x3: 0 to 10

y: up to 1.6×10^{11}

Operators: + − × ÷

Constants: real, uniform(−100, 100)

02 What we changed from Dataset 1

Scale inputs by each variable's max |value|

Protected division: if |denominator| < 10^{-6}, return 1

Drop 13 rows with x3 = 0 (11 train, 2 test): a finite y can't come from ÷ 0

Constant mutation: nudge a constant by ±5 instead of replacing a subtree

Selection, crossover and mutation are unchanged from Dataset 1 · 19,989 train / 4,998 test rows

Dataset 2 · Settings and complexity penalty

Fitness = training MSE + 10^8 × tree size

01 GP settings

100 trees · 50 generations

Initial depth 3 → maximum 5

Early-stop probability: 0.3

Tournament 3 · keep 1 elite

Mutation probability: 0.2

Constant mutation: ± 5

Seeds: 42, 43, 44

Data split: 80 train / 20 test

Diversity: no explicit preservation. 77–94 of 100 trees unique; one dip to 33 that recovered to 90.

02 Complexity penalty

New in Dataset 2: **each tree node adds 10^8** to fitness, so smaller trees win ties

Penalty per node	Avg test MSE
0	5.4×10^{11}
10^7	8.4×10^{11}
10^8 (selected)	**4.7×10^{11}**
5×10^8	5.3×10^{11}

20 generations · 3 seeds · 4 of 7 settings shown

Selected 10^8: best average test MSE. 5×10^8 gave the best single run but was inconsistent

Dataset 2 · A recovered formula

Population 100 · Generations 50 · seed 44

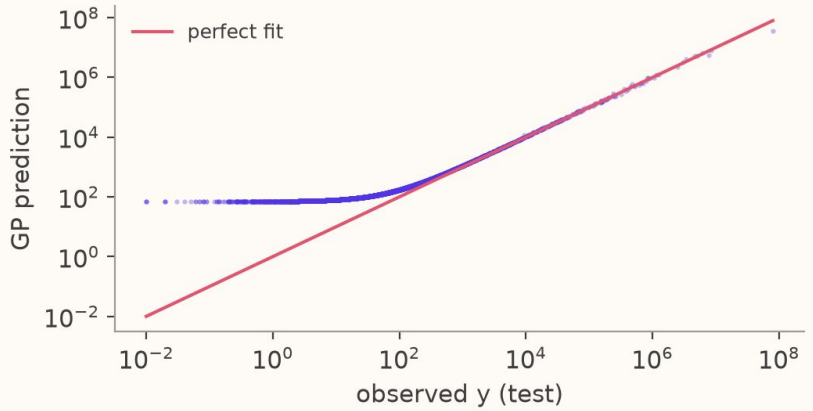
Recovered expression

$$f = 69.36 \cdot x1 \cdot x2 / x3^2 + 69.36$$

Inputs scaled by their max values · 13 nodes

Training MSE	2.88×10^{11}
Test MSE	3.95×10^{11}
Normalized test MSE	0.297

Predicted vs observed (test set)



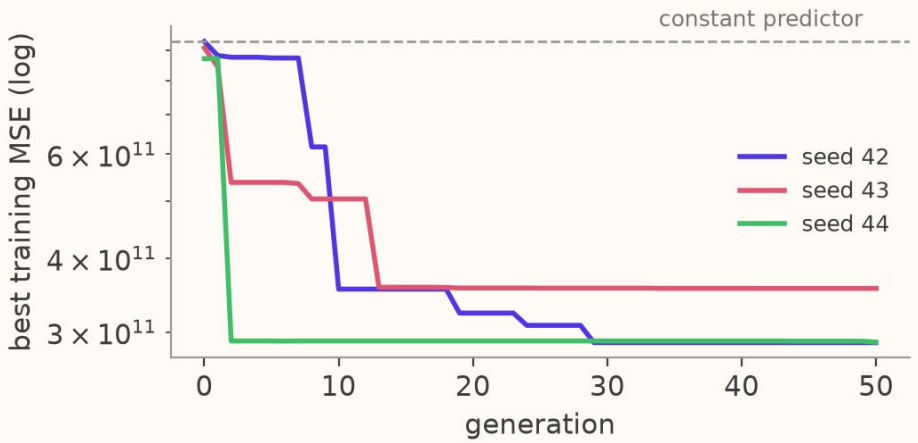
Tracks the law across six orders of magnitude. The + 69 constant shows only below y ≈ 100

$f(x1, x2, x3) = G \cdot x1 \cdot x2 / x3^2$ Newton's gravity, $G \approx 6.9 \times 10^{-11}$ recovered (true 6.674×10^{-11})

Dataset 2 · Stability across seeds

Same settings, three seeds, one fixed 80 / 20 split

01 Convergence



02 Final results

Seed	Norm. test	Nodes	Target
42	0.358	19	Yes
43	0.524	31	No
44	0.297	13	Yes

Average normalized test MSE: **0.393**

2 of 3 seeds recovered $x1 \cdot x2 / x3^2$

Seed 44 converged by generation 2, seed 42 by 29, seed 43 stalled near 3.6×10^{11}

Dataset 2 · Conclusions and next steps

Evaluate structure, stability and the remaining error

Structure

2 of 3

seeds recovered $x1 \cdot x2 / x3^2$
Matches $F = G \cdot m1 \cdot m2 / r^2$

Across seeds

0.393

avg normalized test MSE
Seed 43 stayed complicated

Error floor

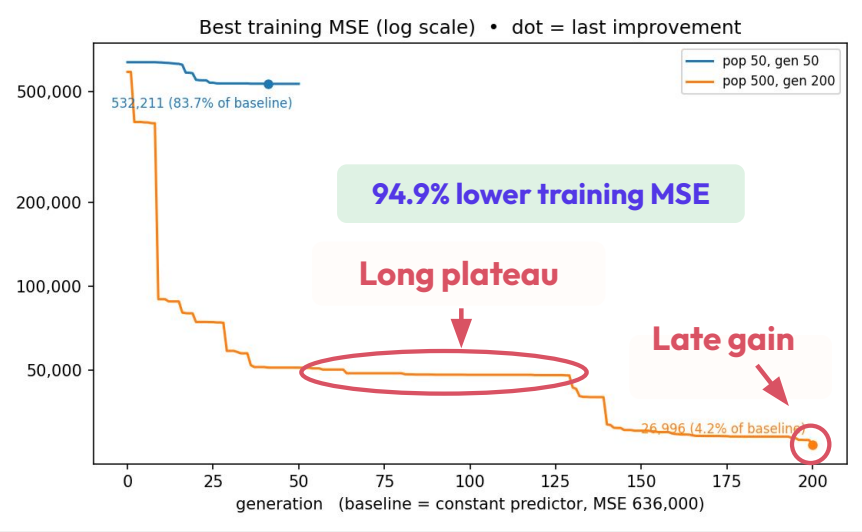
0.309

exact law scores the same
Small x3 rows dominate

Next: fit coefficients directly, test more seeds

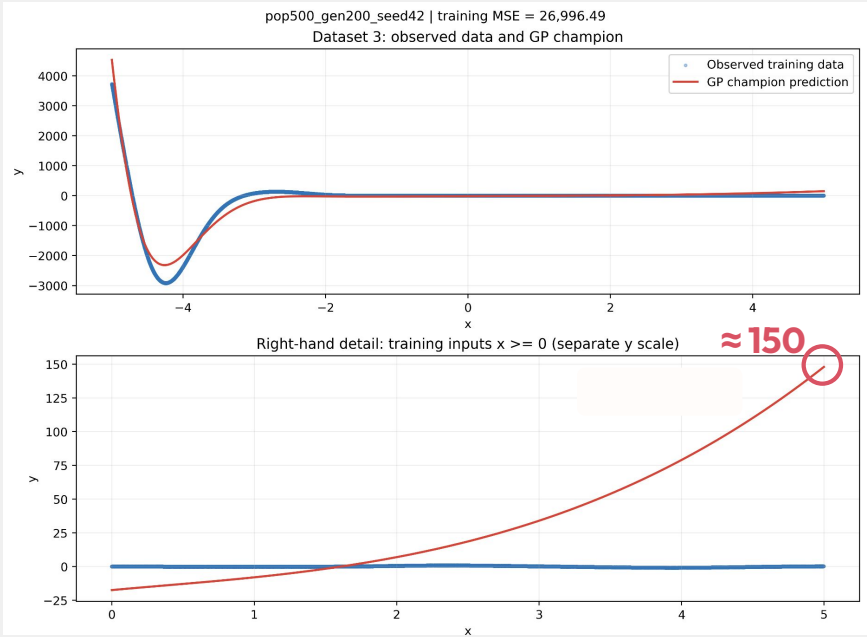
Dataset 3 - First run...

Best training MSE across two budgets



Baseline: random constant predictor MSE: 636,000

Final fit: overview and right-side detail



Search and runtime

Not converging: Still improving at generation 200.

Taking too long: One run took approximately 8 hours.

Remaining fit error

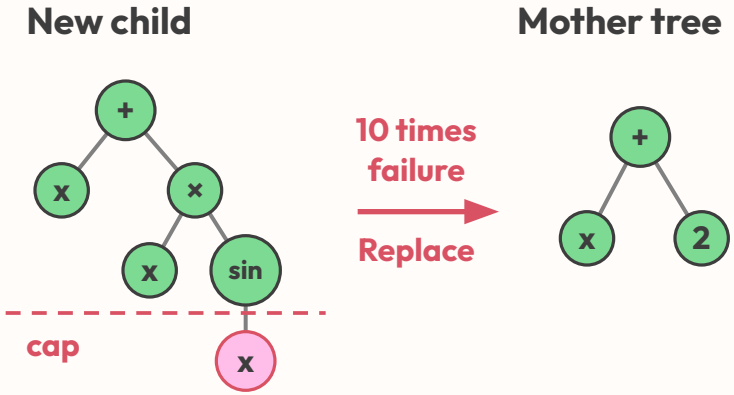
Near x = 5, prediction ≈ 150, while observed y stays near 0.

Early 8-op trial (one seed, gen 50): MSE ≈8.6k vs ≈480k; sin/exp share also changed (40% vs 25%). We retained all 14 ops.

Two hypotheses about useful exploration

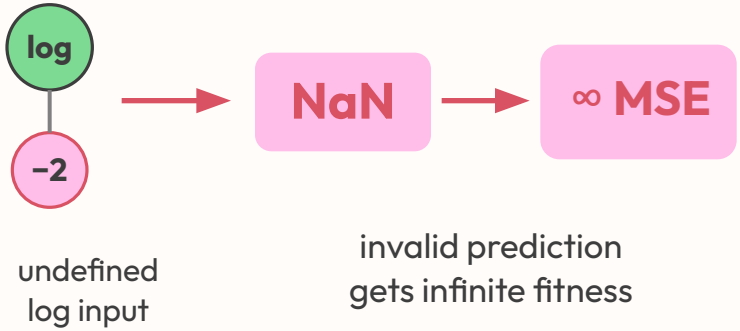
Possible mechanisms to test with population logs

01 Depth cap



Less new structural variation?

02 Invalid predictions



Fewer usable candidates?

Test with logs: offspring rejection and invalid-fitness rates

What I changed in structure:

More room to explore, varied starts and faster evaluation

01 Max_depth

5 → **8**

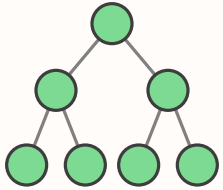
Increase up to 8

More room for complex expressions

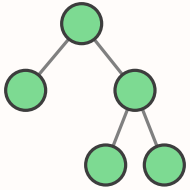
The hard cap remains active

02 Initialization

full



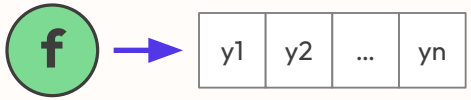
grow



Ramped half-and-half

Mix tree shapes and depths

03 Evaluation



predictions

NumPy

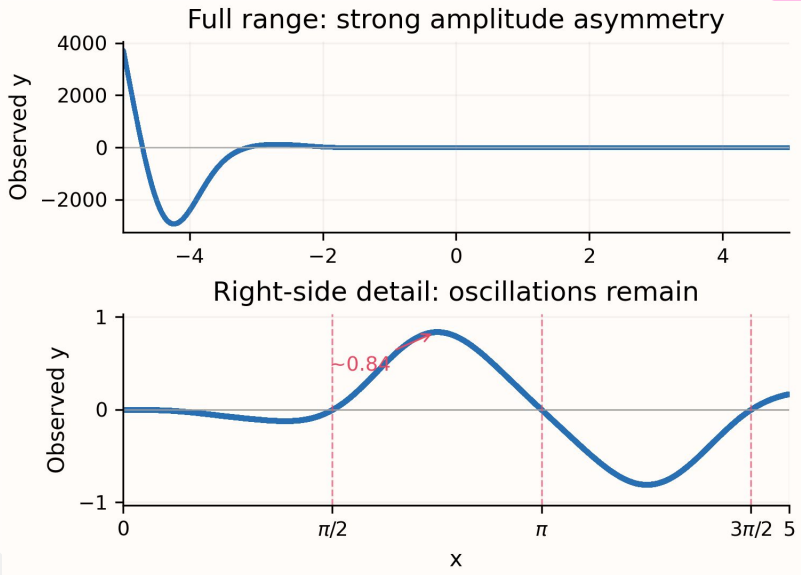
Evaluate training rows as arrays
All in one!

Monitor: string uniqueness, mean tree size and invalid fitness

From data to building blocks

Visual patterns motivate a testable sin / exp sampling bias

01 What the data shows



02 Bias the sampling

All operators stay eligible

+, −, ×, ÷, …

Other operators: weight 1

Sampling weight 2

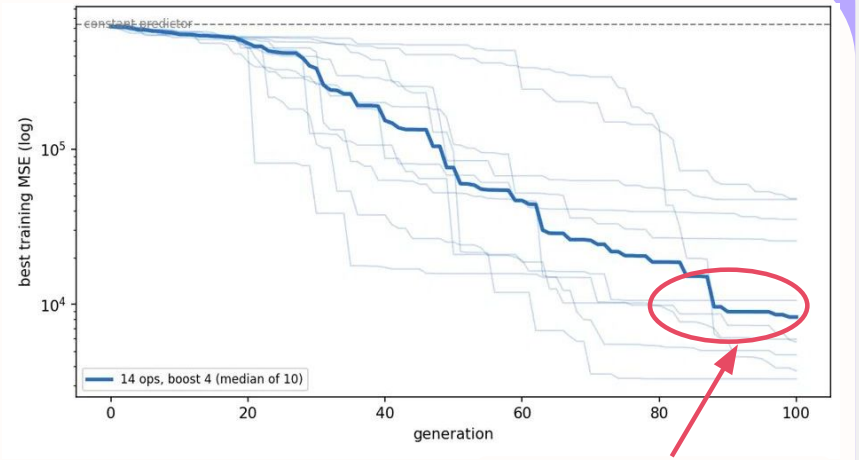
sin **exp**

Hypothesis: high possibilities for sin / exp functions.

Convergence · 100 → 200 generations

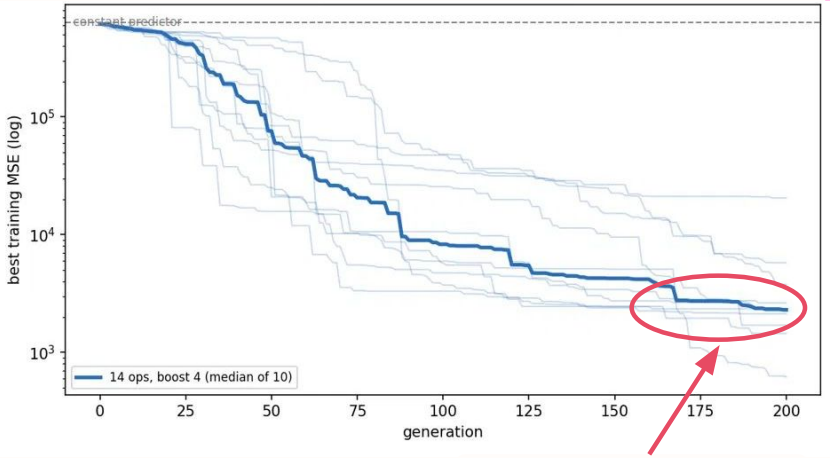
Population 200 · Seeds 1–10 · Boost 2

100 generations



Late drop near gen 90

200 generations



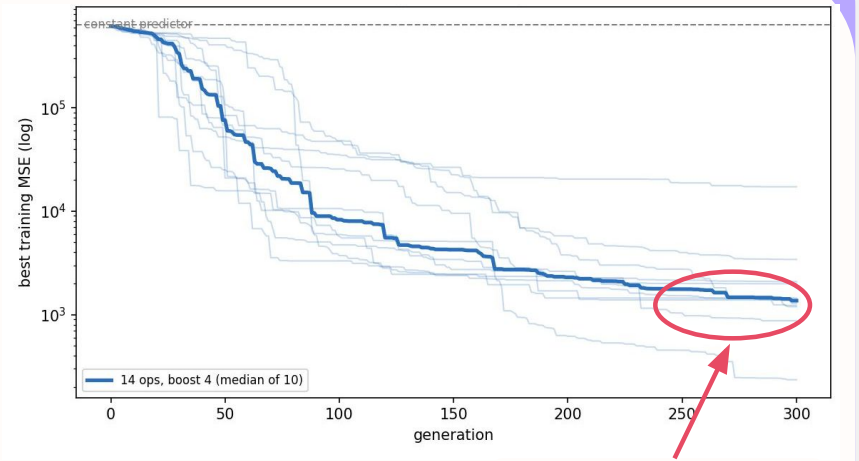
Further late gains

More generations help; final results still vary across seeds

Convergence · 300 → 400 generations

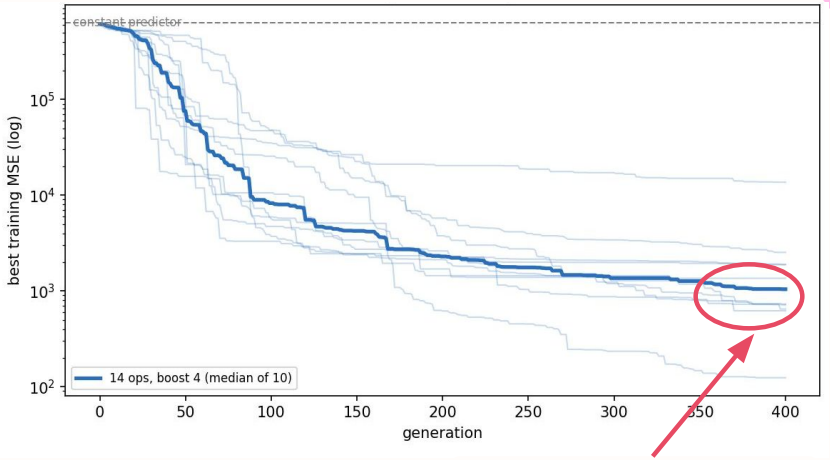
Population 200 · Seeds 1–10 · Boost 2

300 generations



Small late gains

400 generations



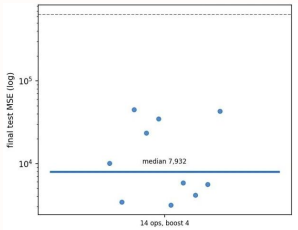
Median nearly flat

Late median gains shrink; seed-to-seed differences remain

Final test MSE across seeds

Population 200 · Seeds 1–10 · Boost 2

100 generations

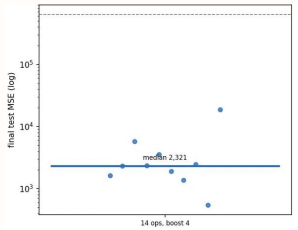


7,932

median test MSE

−70.7%

200 generations

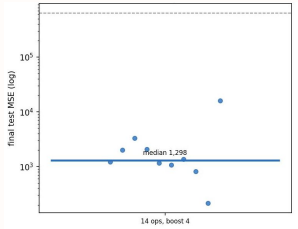


2,321

median test MSE

−44.1%

300 generations

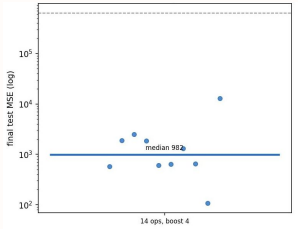


1,298

median test MSE

−24.3%

400 generations



982

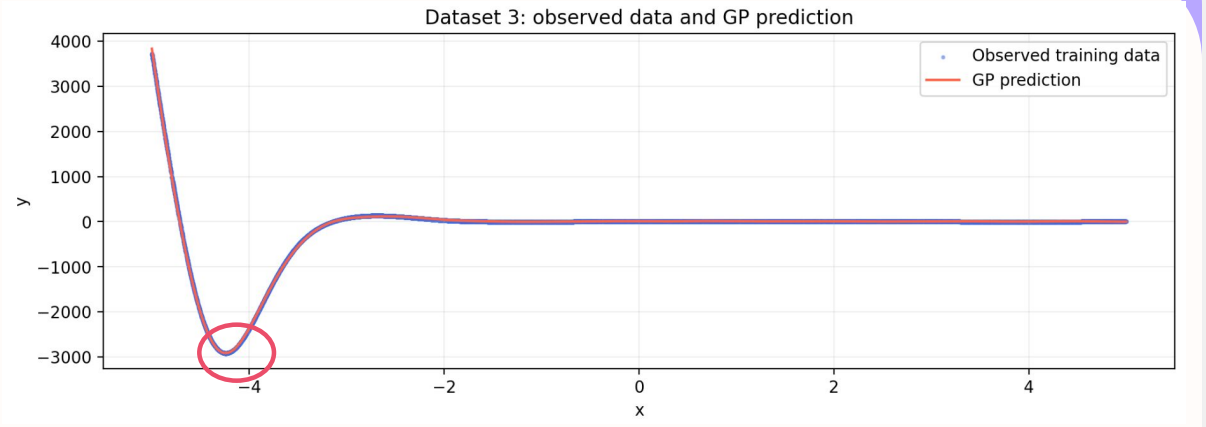
median test MSE

Gen 100 → 400: −87.6% improvement on median test MSE

Selected fit captures the main curve

Boost 2 · Population 200 · Generations 400 · seed 8

Observed data and GP prediction



Main valley and broad shape are closely captured

Fit metrics

Training MSE

123.93

↓96.9% vs 3,952.31

Test MSE

108.05

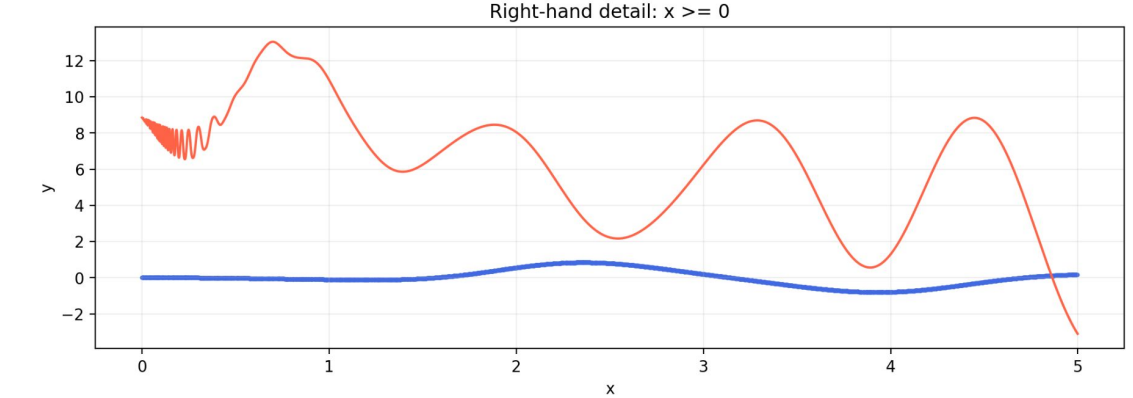
↓97.1% vs 3,698.52

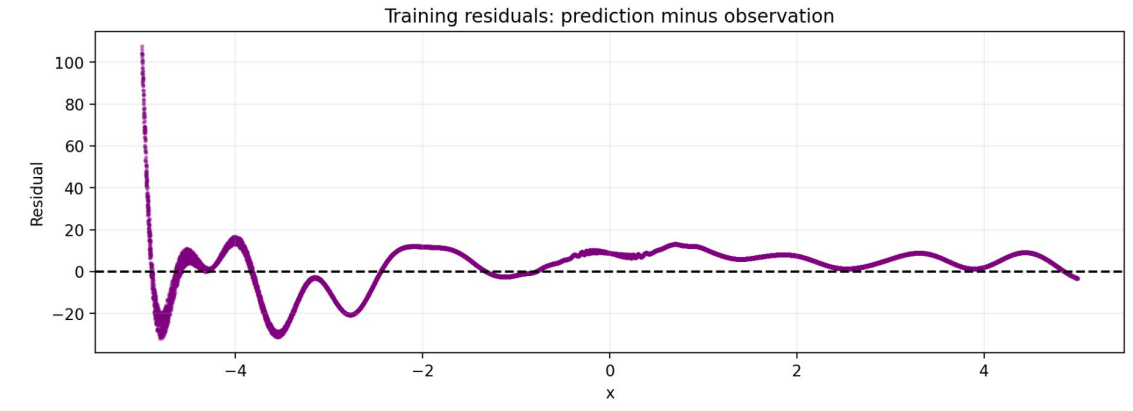
vs P500/G200 · GP seed8
Selected runs · different configurations

Strong overall fit; inspect local errors before claiming uniform accuracy

Good overall fit still has local errors

Boost 2 · Population 200 · Generations 400 · seed 8





01 Right-side detail

Observed y: about −1 to +1
GP prediction: peaks near 13
Mostly overpredicts here

Full-scale overlap hides this gap

02 Residuals

Residual = predicted − observed

Above 0: overprediction
Below 0: underprediction
Waves show systematic mismatch

Structured errors remain

Best recorded expression

Boost 2 · Population 200 · Generations 400 · seed 8

Main term + corrections

Main term	$\mathbf{f(x) = e^{-x} \cdot x \cdot \sin(2x) \cdot [x + \sin(2x)] \cdot [\sin^2 x + \ln(9+x)]}$
Baseline	$- x^2 / [\ln(9+x) + \cos(10)\sin(x)] + 9$
Coupled term	$- \{x + e^{-x}[\sin(2x) + \tfrac{1}{4}\sin(7-x)]\}\cos(5x-\sin 9)$
Nested term	$- \sin(6x)\{\sin[x + \sin(9/x)] + e^{-x}[1+\sin(2x)]\}$

$\cos(10) \approx -0.839 \cdot \sin 9 \approx 0.412 \cdot x = 0$ follows the original protected evaluation

Interpretation checked on the training inputs

Main-term RMS ≈ 810; corrections ≈ 37

Exp modulation + sin(2x) support our sampling hypothesis

Main-only MSE 1,723 → full 123.93

Small corrections still materially improve the fit

Dataset 3 · Conclusions and next steps

Evaluate fit quality, repeated runs and remaining model error

Fit quality

108.05

Test MSE · selected seed 8
Main curve captured
Local errors remain

Across seeds

982

Median test MSE · G400
Typical error decreased
Seed differences remain

Remaining limits

Residual waves

Systematic mismatch
Coefficient precision is a hypothesis to test

Next: real-valued constants, regional error checks or more comprehensive fitness scores!

Separate component effects with matched multi-seed comparisons

Reference

[1] Koza, J. R. *Genetic Programming: Symbolic Regression (quadratic example).* http://www.genetic-programming.com/gpquadraticexample.html

[2] Koza, J. R. (1992). *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press.

[3] Poli, R., Langdon, W. B., & McPhee, N. F. (2008). *A Field Guide to Genetic Programming*. Lulu.com. http://www.gp-field-guide.org.uk

[4] Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362. https://doi.org/10.1038/s41586-020-2649-2

[5] Miller, B. L., & Goldberg, D. E. (1995). Genetic algorithms, tournament selection, and the effects of noise. *Complex Systems*, 9(3), 193–212.

Thank you!